

Daniel Flynn

(201) 956-9271 — danielwflynn@icloud.com — [linkedin.com/in/daniel-flynn-w513](https://www.linkedin.com/in/daniel-flynn-w513) — github.com/danwflynn — danielwflynn.com

Education

Northeastern University, Boston, MA

December 2025

Bachelor of Science in Computer Engineering and Computer Science

Skills

Languages: Python, C/C++, JavaScript/TypeScript, Java, Assembly (x86/ARM/MIPS), Verilog, SQL, LaTeX, Bash
Software: Git, GitHub Actions, CI/CD, Docker, Kubernetes, Netlify, YAML, CMake, Makefile, SystemD, Ansible, SSH
Hardware and Embedded: Linux, STM32, ESP32, Vivado, FPGAs, UART, SPI, I2C, JTAG, DFU, Qt, RTOS
Web Development: Node, React, Vite, HTML/CSS, MongoDB, RESTful APIs, Sockets, OpenAPI, Jest, Cypress

Experience

Digitize

Embedded Software Engineer

April 2026 – Present

- Implemented firmware to control hardware peripherals for RTU microcontrollers in C and assembly for 8051 architecture.
- Wrote custom bit-banged I2C, SPI, and UART drivers for inter-device communication and tested with Saleae logic analyzer.
- Created an integrated circuit for an RTU with an STC89 MCU, WIZnet W5500 ethernet controller, PCF8574 I/O extender, and AHT21 temperature sensor enabling temperature measurements, TCP/UDP communication, and a serial interface.
- Soldered components for a PCB integrating the integrated circuit and an 8 relay module for turning switches on and off.
- Developed a Python Qt GUI for controlling the RTU circuit and gathering data via TCP and UDP socket communication.
- Leveraged OpenSCAD 3d modeling software and Prusa Slicer to 3d print casing and PCB standoffs for RTU circuit board.

NK Labs

Embedded Software Engineering Co-op

January 2025 – June 2025

- Developed software and firmware for a PCR machine using an STM32 microcontroller integrated with a TI AM62 SoC.
- Wrote C++ front-end code with Qt for menus, buttons, experiment graphs/files, and back-end communication over sockets.
- Implemented experiment timers, estimated time calculation, temperature ramping, and event handling on Python back-end.
- Created JavaScript web app to control the PCR machine with socket events by connecting to the device's IP in a browser.
- Built a custom Linux image with Buildroot, integrating front-end and back-end daemons for the PCR machine.

Schneider Electric

Software Engineering Co-op

January 2024 – June 2024

- Wrote C++ code handling server/client communication between edge devices using OPC UA and MQTT protocols.
- Deployed edge computing middleware for industrial automation, orchestrating services with Docker and Kubernetes.
- Cross-compiled edge computing middleware for multiple CPU architectures and integrated WebAssembly modules.
- Contributed to DevOps processes, CI/CD pipeline, and network management using GitHub Actions and Ansible.

Depuy Synthes Mitek Sports Medicine (Johnson and Johnson)

Software Engineering Co-op

January 2023 – June 2023

- Developed software and firmware for the FMS VUE saline and fluid management pump, utilizing a monochrome dot-matrix LCD for the front-end, and IBM Rhapsody for UML design, code organization, simulation, and validation on the back-end.
 - Wrote C code for FMS VUE firmware and Python code to parse information from SystemD logs using journalctl.
 - Used Arm Keil compiler and JTAG debugger to program flash memory, debug firmware code, and create release versions.
 - Implemented auto calibration mode for FMS VUE pressure compensation based on elevation with laboratory testing.
-

Selected Projects

ARM LEGv8 CPU Implementation with Schematic Compiler

November 2025 – January 2026

- Verilog implementation and testbench for a 5-stage pipelined CPU with subset of the ARMv8 instruction set, 64-bit registers, ALU, register file, instruction decoder, program counter, control signals, and branching.
- Python schematic compiler that parses verilog code to generate a directed graph of components using DFS and Graphviz.
- CPU featured in the [Northeastern COE Newsletter](#); Schematic compiler was built as a personal project outside of class.

The Cool Bike Video Game and Engine

January 2026 – Present

- Actively implementing a bike racing video game in C++ by creating a 3d graphics engine and physics engine for it.
- Created an OpenGL rendering pipeline using models, meshes, shaders, and glm linear algebra for camera transformations.
- Implemented collision and gravity physics using hitbox vectors generated from glb 3d models and quaternions for rotations.
- Wrote unit tests for transform logic, camera positioning, and hitbox detection using C++ gtest library for validation.